

Python Cheat Sheet

Table of Contents

1. [Variables](#)
 - [Basic Types](#)
 - [Dict](#)
 - [Tuple](#)
 - [Set](#)
 - [Unpacking](#)
2. [Operators](#)
 - [Comparison](#)
 - [Logical](#)
3. [Conditionals](#)
4. [Loops](#)
 - [for range](#)
 - [for each](#)
 - [while](#)
 - [List Comprehension](#)
 - [Comprehension Filter](#)
 - [enumerate / zip](#)
5. [Functions](#)
 - [Basic Function](#)
 - [Default Args](#)
 - [Lambda](#)
 - [args / kwargs](#)
6. [Classes](#)
 - [Basic Class](#)
 - [Dataclass](#)
 - [Inheritance](#)
 - [Type Hints](#)
 - [Type Alias](#)
 - [Union Type](#)
7. [Exceptions](#)

1. Variables

Basic Types

```
TS:  const name: string = "Alice"  
PY:  name = "Alice"
```

```
name    = "Alice"    # str
age      = 30         # int
height  = 5.9         # float
active  = True        # bool (capital T/F, unlike JS true/false)
nothing = None        # null equivalent

print(type(name))    # <class 'str'> - like typeof
```

Dict

```
TS: const user: Record<string, any> = { name: "Alice", age: 30 }
PY: user = { "name": "Alice", "age": 30 }
```

```
user = {"name": "Alice", "age": 30}

user["name"]           # Alice (KeyError if missing)
user.get("email", "n/a") # safe access with default

user["email"] = "a@b.com" # add / update
del user["age"]           # delete

for key, value in user.items(): # like Object.entries
    print(key, "→", value)

"name" in user            # True (checks keys)

list(user.keys())         # ["name", "email"]
list(user.values())       # ["Alice", "alice@example.com"]

for key in user:          # iterates keys only (like Object.keys)
    print(key, "→", user[key])

merged = {**defaults, **user} # spread (TS: { ...a, ...b })
```

Tuple

```
TS: const point: [number, number] = [1, 2]
PY: point = (1, 2)
```

```
point = (1, 2)
print(point[0])    # 1
print(point[1])    # 2
```

```

x, y = point          # unpack (destructure)

def min_max(nums):
    return min(nums), max(nums)  # multiple return values

lo, hi = min_max([3, 1, 5])

single = (42,)        # single-element needs trailing comma
                        # without comma: (42) == 42, not a tuple

for v in point:        # tuples are iterable
    print(v)

```

Tuples are **immutable** — you cannot change them after creation.

Set

```

TS:  new Set([1, 2, 2, 3])
PY:  {1, 2, 2, 3} → {1, 2, 3}

```

```

nums = {1, 2, 2, 3, 3, 3}
print(nums)          # {1, 2, 3} - duplicates removed

nums.add(4)
nums.discard(1)       # safe remove (no error if missing)

2 in nums             # True - O(1) lookup

unique = list(set(words))  # deduplicate a list

a | b    # union
a & b    # intersection
a - b    # difference

empty = set()         # NOT {} - that creates an empty dict

```

Unpacking

```

TS:  const [a, b] = [1, 2]
PY:  a, b = [1, 2]

```

```

a, b = [1, 2]

a, b = b, a          # swap without temp variable

first, *rest = [1, 2, 3, 4, 5]  # rest = [2, 3, 4, 5]
*head, last = [1, 2, 3, 4, 5]  # head = [1, 2, 3, 4]

x, _, z = (10, 20, 30)  # ignore with _

for num, letter in [(1, "a"), (2, "b")]: # unpack in loop
    print(num, letter)

```

2. Operators

Comparison

```

TS:  === (strict, no coercion)
PY:  ==  (always strict – no === needed)

```

```

1 == 1      # True
1 == "1"    # False – no type coercion (unlike JS ==)
1 != 2      # True   (TS: !==)
3 > 2       # True
3 >= 3      # True

```

Logical

```

TS:  &&  ||  !
PY:  and or  not

```

```

x and y      # False (TS: x && y)
x or y       # True  (TS: x || y)
not x        # False (TS: !x)

```

3. Conditionals

```

TS:  if (x > 0) { } else if (x === 0) { } else { }
PY:  if x > 0: / elif x == 0: / else:

```

– no parens, no braces, colon + indent

```
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")
```

4. Loops

for range

```
TS: for (let i = 0; i < 5; i++)
PY: for i in range(5):
```

```
for i in range(5):      # 0 1 2 3 4
for i in range(1, 6):   # 1 2 3 4 5 (upper bound excluded)
for i in range(0, 10, 2): # 0 2 4 6 8 (step)
```

for each

```
TS: for (const fruit of fruits)
PY: for fruit in fruits:
```

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

while

```
TS: while (i < 5) { i++ }
PY: while i < 5: (no i++, use i += 1)
```

```
i = 0
while i < 5:
    print(i)
    i += 1
```

List Comprehension

```
TS: arr.map(x => x * 2)
PY: [x * 2 for x in arr]
```

```
result = [x * 2 for x in arr]
```

Comprehension Filter

```
TS: arr.filter(x => x % 2 === 0).map(x => x * 10)
PY: [x * 10 for x in arr if x % 2 == 0]
```

```
evens          = [x for x in nums if x % 2 == 0]
doubled_evens = [x * 2 for x in nums if x % 2 == 0]

lengths = {w: len(w) for w in words}    # dict comprehension
squares = {x ** 2 for x in nums}        # set comprehension

flat = [n for row in matrix for n in row] # flatten 2D list
```

enumerate / zip

```
TS: arr.forEach((item, i) => ...)
PY: for i, item in enumerate(arr):
```

```
for i, fruit in enumerate(fruits):    # 0-based index
for i, fruit in enumerate(fruits, start=1): # 1-based index

for fruit, price in zip(fruits, prices): # loop two lists in parallel
# zip stops at shortest list
```

```
# combine enumerate + zip
for i, (fruit, price) in enumerate(zip(fruits, prices), start=1):
    print(f"{i}. {fruit} = ${price}")
```

5. Functions

Basic Function

```
TS: function foo(a: number): number { return a }
PY: def foo(a): return a
    - no types required, no braces, colon + indent
```

```
def foo(a):
    return a
```

Default Args

```
TS: function greet(name: string = "World"): string
PY: def greet(name="World"):
```

```
def greet(name="World"):
    return f"Hi {name}"      # f-string = template literal

greet()                    # Hi World
greet("Alice")             # Hi Alice
```

Lambda

```
TS: (x: number) => x * 2
PY: lambda x: x * 2
```

```
double = lambda x: x * 2
add     = lambda x, y: x + y

sorted(words, key=lambda w: len(w))      # sort by length
sorted(people, key=lambda p: p["age"])   # sort by dict key
```

```
list(map(lambda x: x * 2, nums))      # map
list(filter(lambda x: x % 2 == 0, nums))  # filter
```

Prefer comprehensions over `map/filter` in most cases.

args / kwargs

```
TS: function foo(...args: number[])
PY: def foo(*args):
```

```
def add(*args):          # collects as tuple
    return sum(args)

def describe(**kwargs):  # collects as dict
    for key, value in kwargs.items():
        print(f"{key} = {value}")

def mixed(required, *args, **kwargs):
    pass

add(*[1, 2, 3])          # unpack list into positional args
describe(**{"name": "Alice"}) # unpack dict into keyword args
```

6. Classes

Basic Class

```
TS: class Dog { constructor(public name: string) {} }
PY: class Dog:
    def __init__(self, name): self.name = name
```

```
class Dog:
    def __init__(self, name):  # __init__ = constructor
        self.name = name      # self = this

    def bark(self):
        print(f"{self.name} says: Woof!")

    def __repr__(self):        # like toString()
        return f"Dog({self.name!r})"
```

```
d = Dog("Rex")
d.bark()
```

Dataclass

```
TS: type User = { name: string; age: number }
PY: @dataclass
    class User:
        name: str
        age: int
```

```
from dataclasses import dataclass

@dataclass
class User:
    name: str
    age: int

u = User(name="Alice", age=30)
print(u)          # User(name='Alice', age=30) - __repr__ auto-generated
print(u.name)     # Alice
```

Inheritance

```
TS: class Dog extends Animal { constructor(...) { super(...) } }
PY: class Dog(Animal):
    def __init__(self, ...): super().__init__(...)
```

```
class Animal:
    def __init__(self, name: str):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

    def __repr__(self):
        return f"{self.__class__.__name__}({self.name!r})"

class Dog(Animal):
    def speak(self):          # override
        return f"{self.name} says: Woof!"
```

```

class Cat(Animal):
    def __init__(self, name: str, indoor: bool):
        super().__init__(name) # call parent constructor
        self.indoor = indoor

    def speak(self):
        return f"{self.name} says: Meow!"

animals = [Dog("Rex"), Cat("Whiskers", indoor=True), Animal("???")]
for animal in animals:
    print(animal.speak())

isinstance(d, Dog)      # True (like isinstance)
isinstance(d, Animal)   # True - works up the chain
type(d).__name__        # "Dog"
repr(d)                 # "Dog('Buddy')" - uses __repr__

```

Type Hints

```

TS: interface Foo { bar: string }
PY: class Foo:
    bar: str    - hints are optional, NOT enforced at runtime

```

```

class Foo:
    bar: str

    def __init__(self, bar: str) -> None:
        self.bar = bar

f = Foo("hello")
print(f.bar)           # hello
print(type(f.bar))     # <class 'str'>

```

Type Alias

```

TS: type ID = string
PY: type ID = str    - Python 3.12+

```

```

type ID = str

user_id: ID = "abc-123"

```

Union Type

```
TS: type ID = string | number
PY: type ID = str | int    - Python 3.10+
```

```
type ID = str | int

def print_id(id: ID):
    print(id, "→", type(id).__name__)
```

7. Exceptions

```
TS: try { } catch (e) { } finally { }
PY: try: / except ExceptionType as e: / else: / finally:
```

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("Caught:", e)

# multiple except blocks
try:
    return int(value)
except ValueError:
    print("bad value")
except TypeError:
    print("bad type")

# else – runs only if NO exception was raised
try:
    x = int("42")
except ValueError:
    print("bad input")
else:
    print("OK:", x)

# finally – always runs (cleanup)
try:
    risky()
except ValueError as e:
    print("Error:", e)
finally:
    print("always runs")
```

```
# raise - with custom message
def divide(a, b):
    if b == 0:
        raise ValueError("b must not be zero")
    return a / b

try:
    divide(5, 0)
except ValueError as e:
    print("Raised:", e)
```

Common exceptions:

Exception	When
KeyError	dict key not found
IndexError	list index out of range
TypeError	wrong type passed
ValueError	right type, bad value
AttributeError	object has no such attribute
FileNotFoundError	file doesn't exist